

System Design Prerequisites – Scaling Foundations

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 4 Jan 2027 – 6 Jan 2027 (Days 141–143)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm & 9:00–10:00pm** · Sat–Sun **9:00am–12:00pm, 3:00–5:30pm, 6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import <calendar/mern-mastery-study.ics>

Phase 7 start — Days 141–143 (BEFORE full HLD designs).

Prerequisites: Networking (10b), Databases (06), Auth (07).

Unlocks: HLD (10), Distributed Systems (11).

Why this exists (don't skip)

[10-system-design-complete.pdf](#) jumps into "Design Instagram" but assumes you understand:

- Vertical vs horizontal scaling
- Load balancer algorithms
- Caching layers
- Database read replicas vs sharding
- Async processing with queues

This PDF builds the **foundation layer** before full system designs.

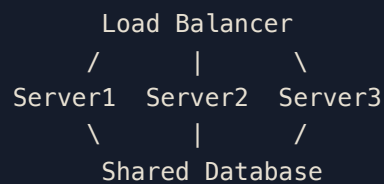
1. Scaling 101

Vertical scaling (scale up)

- Bigger machine: more CPU, RAM, SSD
- **Pros:** Simple, no code changes
- **Cons:** Hardware limits, single point of failure, expensive
- **When:** Early stage, < 10K users

Horizontal scaling (scale out)

- More machines behind load balancer
- **Pros:** No upper limit, fault tolerance
- **Cons:** Stateless app required, data consistency harder
- **When:** Production at scale



2. Load balancing

Algorithms

Algorithm	Behavior	Best for
Round Robin	Rotate sequentially	Equal servers
Weighted RR	More traffic to powerful servers	Mixed hardware
Least Connections	Route to least busy	Varying request times
IP Hash	Same client → same server	Session stickiness
Consistent Hash	Minimal remap on add/remove	Cache shards

Layer 4 vs Layer 7

- **L4 (TCP):** Routes by IP/port — fast, no HTTP awareness
- **L7 (HTTP):** Routes by URL path, headers — SSL termination, path-based routing

Tools: NGINX, HAProxy, AWS ALB, Cloudflare.

3. Caching layers

Browser cache → CDN → Application cache (Redis) → Database

↑	↑	↑	↑
fastest	fast	moderate	slowest
smallest	...	...	source of truth

Cache strategies

Strategy	How	Use when
Cache-aside	App checks cache, on miss reads DB, writes cache	General purpose
Write-through	Write to cache + DB together	Strong consistency needed
Write-behind	Write cache, async flush to DB	Write-heavy, can tolerate delay
Read-through	Cache fetches from DB on miss	Simplify app code

Cache invalidation

- **TTL:** Simple, stale data possible
- **Event-driven:** Invalidate on write — more accurate
- **"Two hard problems":** Naming things, cache invalidation, off-by-one errors

Redis use cases

- Session store
- Rate limiting counters
- Leaderboards (sorted sets)

- Pub/sub for real-time
- Distributed locks

4. Database scaling (before sharding)

Step 1: Optimize queries

- Indexes on WHERE/JOIN columns
- EXPLAIN ANALYZE
- Fix N+1 queries

Step 2: Read replicas

```
Writes → Primary DB
      |
Reads ← Replica 1, Replica 2, Replica 3
```

- **Replication lag:** Reads may be stale (eventual consistency)
- **Use:** Read-heavy workloads (feeds, profiles)

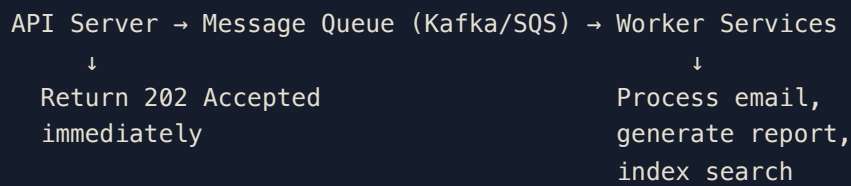
Step 3: Connection pooling

- PgBouncer for PostgreSQL
- Pool size tuning

Step 4: Sharding (last resort)

- Split data across DB instances by shard key (user_id, geo)
- **Problem:** Cross-shard joins, rebalancing

5. Async processing



Why: Decouple slow operations, absorb traffic spikes, retry failed jobs.

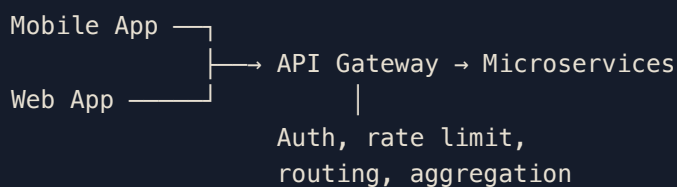
Patterns:

- Task queue (Celery, BullMQ)
- Event bus (Kafka topics)
- Webhook callbacks

6. CDN (Content Delivery Network)

- Edge servers worldwide cache static assets
- Reduces latency, offloads origin server
- **Cache:** JS, CSS, images, videos
- **Providers:** CloudFront, Cloudflare, Fastly

7. API gateway / BFF



Backend for Frontend (BFF): Separate API layer per client type (mobile vs web).

8. Monitoring basics (before K8s PDF)

Type	What	Tools
Metrics	CPU, QPS, latency p99	Prometheus, Grafana
Logs	Structured JSON logs	ELK, CloudWatch
Traces	Request flow across services	Jaeger, Datadog
Alerts	SLO breach → page on-call	PagerDuty

SLI / SLO / SLA

- **SLI:** Metric (e.g., request latency p99)
- **SLO:** Target (e.g., p99 < 200ms for 99.9% of requests/month)
- **SLA:** Contract with penalties
- **Error budget:** Allowed downtime = 100% - SLO

9. Reliability patterns

Pattern	Purpose
Retry with exponential backoff	Transient failures
Circuit breaker	Stop calling failing service
Bulkhead	Isolate failures (thread pools)
Timeout	Don't wait forever
Idempotency keys	Safe retries for payments
Health checks	LB removes unhealthy nodes

10. Estimation template (use in every HLD)

```
DAU = 10 million
Actions per user per day = 20
QPS = (10M × 20) / 86400 ≈ 2,300 average
Peak QPS = 3–5× average ≈ 10,000

Storage per object = 1 KB
New objects/day = 1M
Storage/day = 1 GB
Storage/year ≈ 365 GB (+ replicas)
```

11. Progression to full HLD

After this PDF, you can understand:

- URL shortener → caching + DB + redirect
- Feed → fan-out + cache + CDN
- Chat → WebSocket + queue + storage
- Payment → idempotency + saga + ACID

Next PDF: [10-system-design-complete.pdf](#) (full designs)

12. Interview Q&A

Q: How do you scale a read-heavy application?

A: Start with query optimization and indexes. Add Redis cache for hot data with cache-aside pattern. Add read replicas for database reads. Use CDN for static/semi-static content. If still insufficient, consider sharding by read patterns or search engine (Elasticsearch) for complex queries. Monitor cache hit rate and replication lag.

Q: Cache-aside vs write-through?

A: Cache-aside: application manages cache — read cache, on miss read DB and populate cache, on write update DB and invalidate cache. Flexible, cache only hot data. Write-through: write goes to cache and DB synchronously — cache always warm but writes slower and cache filled with data that may never be read. Choose cache-aside for most web apps; write-through when reads must always hit cache.

Q: When do you shard a database?

A: When vertical scaling and read replicas insufficient — typically write throughput exceeds single node capacity (often 10K+ writes/sec depending on workload) or storage exceeds manageable size (TB+). Shard key must align with query patterns to avoid cross-shard operations. Plan for rebalancing and operational complexity. Many companies delay sharding via aggressive caching and read replicas.

13. Gate to HLD (PDF #10)

- ☐ Draw 4-layer caching diagram from memory
 - ☐ Explain read replica lag tradeoff
 - ☐ Name 3 load balancer algorithms
 - ☐ Calculate QPS from DAU estimate
 - ☐ Explain why message queue for email sending
-

System Design Prerequisites Interview – 50 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

This bank covers caching, load balancing, the CAP (Consistency, Availability, Partition tolerance) theorem, and back-of-the-envelope estimation—foundational topics for system design interviews at product companies. Spell out abbreviations on first use.

1. Why are system design prerequisites important before HLD problems?

Caching, load balancing, CAP theorem, and estimation appear in every large design. Without them you hand-wave performance. Prerequisites give vocabulary to justify Redis, replicas, and partition tolerance. Interviewers at Amazon and Google expect estimation within order of magnitude early in the session.

2. What is cache-aside (lazy loading) pattern?

Application reads cache first; on miss loads database, populates cache, returns. Updates invalidate or update cache entry. Simple and common. Risk: cache stampede on expiry — many requests hit database. Mitigate with probabilistic early expiry or single-flight locking.

3. Explain read-through vs write-through cache.

Read-through: cache library loads from database on miss transparently. Write-through: write goes to cache and database synchronously — strong consistency, higher write latency. Write-back: write cache only, flush async — faster, data loss risk on crash. Pick based on consistency tolerance.

4. What is cache eviction policy LRU vs LFU?

LRU (Least Recently Used) evicts stale by recency — good for general temporal locality. LFU (Least Frequently Used) keeps popular items — good for skewed hot sets. Redis supports multiple policies via `maxmemory-policy`. Tiny cache + power law traffic favors LFU or TTL-heavy.

5. How do you size a cache?

Estimate working set: unique keys accessed in time window $\times$ value size. Hit rate target 90%+ for hot path. Monitor miss rate and P99 latency. Over-provision memory for eviction headroom. Cost vs database load saved — one Redis cluster cheaper than 10x read replica load for read-heavy key-value.

6. What problems do caches introduce?

Stale data, cache penetration (queries for nonexistent keys), cache avalanche (mass expiry), hot keys, inconsistency on writes. Thundering herd on popular miss. Document invalidation strategy per entity. Never cache without TTL unless immutable content.

7. Explain consistent hashing for distributed cache.

Keys and nodes map to hash ring; key assigned to next clockwise node. Adding/removing node moves only adjacent key range — minimal reshuffle vs mod N. Virtual nodes improve balance. Used in Memcached clusters and Dynamo-style stores. Mention as scaling cache beyond single Redis instance.

8. Layer 4 vs Layer 7 load balancing?

Layer 4 (Transport): routes by IP and port — fast, no HTTP parsing. Layer 7 (Application): inspects headers, path, cookies — sticky sessions, path-based routing to microservices. AWS Application Load Balancer is L7; Network Load Balancer is L4. TLS termination often at L7 load balancer.

9. What load balancing algorithms should you know?

Round robin: equal rotation. Weighted round robin: capacity differences. Least connections: favor idle server. IP hash: sticky approximate. Random with two choices: power of two choices theory. Health checks exclude unhealthy hosts. Geo routing sends to nearest region.

10. What is a reverse proxy vs forward proxy?

Reverse proxy sits before servers — clients hit nginx protecting origin; SSL termination, caching, routing. Forward proxy sits before clients — corporate egress filter. In system design, reverse proxy

and load balancer roles overlap. CDN edge acts as reverse proxy cache globally.

11. Explain DNS load balancing and geoDNS.

DNS returns different IP addresses based on client location or weighted policy. Low TTL enables failover shift. Not fine-grained — client caches DNS. GeoDNS routes India users to Mumbai data center. Combine with anycast IP for fast failover. First hop in global routing strategy.

12. What is sticky session and when avoid it?

Load balancer pins user to same backend via cookie or IP hash. Needed for in-memory session state. Avoid when possible — store session in Redis for stateless servers enabling even load and graceful deploy. Sticky hides uneven load if users vary in activity.

13. CAP theorem explained for interviews.

Consistency: all nodes see same data simultaneously. Availability: every request gets non-error response. Partition tolerance: system operates despite network splits. Pick two under partition — CP or AP. CA only on single node. Real systems tune per operation — not one global choice.

14. PACELC theorem extension?

If Partition then A or C else Latency or Consistency. Even without partition, distributed databases trade latency for consistency — synchronous replication slower. DynamoDB and Cassandra document tunable consistency per read. Shows deeper understanding beyond slogan CAP.

15. Strong vs eventual consistency examples?

Strong: linearizable read after write on primary relational database. Eventual: Cassandra write quorum, read may be stale briefly. Causal consistency middle ground for social feeds. Session consistency sticks reads to user's writes. Name consistency level in design explicitly.

16. What is quorum read/write in distributed storage?

Replicate to N nodes; write succeeds if W nodes acknowledge; read R nodes; if $R + W > N$ overlap guarantees latest version if single writer. Tune W and R for latency vs durability. Used in Dynamo,

Cassandra, Riak. Explain during distributed database deep dives.

17. Back-of-envelope: requests per second from daily active users?

DAU (Daily Active Users) 10M, each 50 requests/day $\rightarrow 500\text{M/day} \div 86400 \approx 5800$ RPS (Requests Per Second) average. Peak 3-5x average $\rightarrow \sim 20\text{-}30\text{K}$ RPS plan. Show multiplication aloud; round aggressively. Interviewers accept order-of-magnitude correctness.

18. Estimate storage for photo app.

10M users upload 2 photos/week, 2 MB average $\rightarrow 20\text{M photos/week} \times 2 \text{ MB} = 40 \text{ TB/week raw}$; with 3 thumbnail variants $\times 0.3 \rightarrow \sim 52 \text{ TB/week}$. Annual multiplies — compression and dedup reduce. Drives object storage not block storage choice and lifecycle policies to cold tier.

19. Estimate bandwidth for video streaming.

1M concurrent viewers $\times 5 \text{ Mbps bitrate} = 5 \text{ Tbps aggregate}$ — CDN mandatory. Origin serves fraction on cache miss. Peak concurrent fraction of DAU matters. Mention 80% cache hit dramatically cuts origin egress cost on AWS bill.

20. Latency numbers every designer should know?

L1 cache $\sim 1 \text{ ns}$, RAM $\sim 100 \text{ ns}$, SSD $\sim 100 \mu\text{s}$, cross-region RTT (Round Trip Time) $\sim 100 \text{ ms}$, read 1 MB from disk $\sim 20 \text{ ms}$. Network dominates distributed systems. Saying 'add Redis saves 10 ms database round trip' grounds design choices credibly.

21. What is CDN cache hit ratio impact?

90% hit: 10% traffic hits origin. 99% hit: 1% origin — 10x difference in origin capacity. Static assets long TTL; API short or no cache. Cache bust via fingerprinted filenames in frontend builds. Measure hit ratio in CloudFront metrics.

22. Compare Redis vs Memcached.

Redis: rich types, persistence optional, replication, pub/sub — general purpose. Memcached: simple multithreaded key-value, pure cache. Redis single-threaded per instance but cluster scales out. Choose Redis for sorted sets, leaderboards, rate limits; Memcached pure LRU blob cache.

23. What is database connection pooling?

Reuse TCP connections to database instead of open per request — saves handshake latency and file descriptors. Pool size tuned to CPU cores and database max connections. Too large pool contends database; too small queues app threads. PgBouncer, HikariCP standard.

24. Read replica scaling limits?

Replication lag grows with write rate; replicas share disk I/O on primary for binlog shipping. Reads still hit connection limits. Cannot scale writes via replicas — only sharding. Cache absorbs read scale first; replicas second; shard when both insufficient.

25. What is sharding vs partitioning?

Partitioning splits table within one database engine — range or hash partitions for manageability. Sharding splits across independent database servers — horizontal scale. Sharding adds application routing complexity. Start partitioned single DB; shard when single node maxed.

26. Hot partition problem in DynamoDB?

Poor partition key — all writes same key — throttles single partition 1000 WCU limit. Design key with high cardinality — append random suffix to celebrity user events. Adaptive capacity temporarily helps but fix schema. Mention in any wide-column design.

27. What is autoscaling trigger design?

Scale on CPU, request rate, queue depth, or custom metric — latency SLO breach. Cooldown prevents flapping. Pre-warm before known events — Flipkart sale. Scale database differently — read replicas lag behind; prefer cache and queue absorb before adding DB write capacity.

28. Explain thundering herd problem.

Many clients simultaneously miss cache or retry failed service — overload backend. Fixes: request coalescing, jittered exponential backoff, circuit breaker, pre-warm cache, stagger TTL expiry. Classic when celebrity posts and millions refresh feed same second.

29. What is single point of failure (SPOF)?

Component whose failure stops entire system — single database without replica, single load balancer without pair. Eliminate via redundancy, failover, multi-AZ. Identify SPOF in diagram proactively — interviewers reward without prompting. DNS and control plane can be SPOF too.

30. Active-passive vs active-active failover?

Active-passive: standby takes over on failure — simpler, wasted capacity idle. Active-active: both serve traffic — harder data sync, better utilization. RTO (Recovery Time Objective) and RPO (Recovery Point Objective) drive choice. Banking may accept minutes RTO; social feed seconds.

31. What is RPO and RTO?

RPO: max acceptable data loss measured in time — backups every 5 min → 5 min RPO. RTO: max downtime to restore service. Drive backup frequency and multi-region investment. State assumptions in disaster recovery section of design.

32. How estimate memory for Redis session store?

$50\text{M sessions} \times 2\text{ KB session blob} = 100\text{ GB}$ plus overhead 30% → ~130 GB cluster. Replication doubles for primary-replica. Plan shard if exceeds single node RAM. Eviction policy volatile-lru if session TTL anyway.

33. What is write amplification in storage?

One logical write causes multiple physical writes — SSD garbage collection, LSM-tree compaction in Cassandra/RocksDB. Affects disk lifespan and latency spikes. Choose storage engine aware of access pattern — append-only logs vs update-heavy B-trees.

34. Explain leader-follower database replication.

Leader accepts writes; followers replay binlog asynchronously or semi-sync. Follower promotion on leader failure — automatic failover with Raft or manual. Split-brain if two leaders — use quorum and fencing. Read scaling on followers with lag caveat.

35. Multi-master replication pitfalls?

Concurrent writes same row conflict — need conflict resolution, last-write-wins dangerous for counters. Use CRDT or application merge. Higher write availability across regions. MySQL group replication or Postgres logical replication advanced. Prefer single leader per shard simpler.

36. What is denormalization and when?

Duplicate data to avoid joins at read time — store username on tweet row. Trade storage and update complexity for read speed. Eventual consistency on duplicated fields via async workers. Common in read-heavy feeds and analytics rollups.

37. Indexing trade-offs in high-write tables?

Each index slows inserts and updates — must maintain B-tree. Over-indexing kills write throughput. Index only query filters — `(user_id, created_at)` for timeline. Covering index avoids table lookup. Monitor slow query log.

38. What is connection timeout vs request timeout?

Connection timeout: TCP establish fails fast if host dead. Request timeout: total wait including processing. Set both in clients and load balancers. Cascading timeouts should decrease at each hop — outer 30s, inner service 5s — fail fast.

39. Compare vertical partitioning vs horizontal?

Vertical: split columns — hot profile fields separate table from blob bio. Horizontal: split rows by key — sharding. Vertical reduces row width I/O; horizontal scales size. Use vertical first when wide rows; horizontal when row count explodes.

40. What is gossip protocol in distributed systems?

Nodes periodically exchange state with random peers — membership, failure detection, metadata. Cassandra and Consul use gossip. Eventually all nodes converge without central coordinator. Scales $O(\log N)$ rounds but eventual. Mention for decentralized cluster management.

41. Explain heartbeat and health checks.

Clients or load balancers send periodic heartbeat; miss N beats marks unhealthy. False positive from GC pause — use multiple checks. Graceful shutdown deregisters before stop. Kubernetes liveness vs readiness — readiness removes from service during startup.

42. What is backpressure?

Slow consumer signals producer to slow — bounded queue drops or blocks. Prevents OOM (Out Of Memory) when downstream overloaded. Kafka consumer pause, reactive streams, HTTP 503 with Retry-After. Essential in pipeline design without unbounded buffers.

43. Estimate QPS for API from MAU?

MAU (Monthly Active Users) 100M, 10% DAU → 10M DAU. Each 20 API calls/day → 200M calls/day ÷ 86400 ≈ 2300 RPS average, peak 5x ~12K RPS. Adjust for batch vs interactive. Shows structured thinking.

44. What is three-tier architecture?

Presentation (web/mobile), application (business logic), data (database). Classic starting diagram. Evolved to microservices and BFF (Backend for Frontend) but still valid skeleton. Load balancer fronting stateless app tier remains pattern.

45. Compare push CDN vs pull CDN origin?

Push: upload content to CDN proactively — predictable assets. Pull: CDN fetches from origin on first request — user-generated content. Most UGC uses pull with TTL. Origin shield reduces load on single origin datacenter.

46. What is service discovery?

Dynamic registry of service instances — Consul, etcd, Kubernetes DNS. Clients resolve `payments-service` to healthy pod IPs. Replaces static load balancer config. Health check integration removes crashed instances. Required for autoscaling microservices.

47. Explain bulkhead pattern for resource isolation.

Separate thread pools or connection limits per dependency — one slow service cannot exhaust all threads. Named after ship compartments. Netflix Hystrix bulkheads. Pair with timeouts. Simple version: dedicated Redis cluster per critical vs best-effort cache.

48. What is SLA vs SLO vs SLI?

SLI (Indicator): measured metric — P99 latency. SLO (Objective): target — P99 < 200ms 99.9% of month. SLA (Agreement): contractual consequence if SLO missed — credits. Error budget = allowed unreliability. Google SRE culture core vocabulary.

49. How plan capacity for 10x growth?

Identify first bottleneck — usually database writes or hot service CPU. Horizontal scale stateless tier cheaply. Shard data before vertical limits. Load test 2x expected peak quarterly. Cost model: linear vs superlinear growth. Deferred optimization with metrics-driven triggers.

50. Summarize prerequisite checklist before any HLD.

Estimate RPS and storage; draw load balancer, app, cache, database, queue; state consistency model; identify SPOF; plan cache strategy and invalidation; mention monitoring. Run mentally in two minutes before diving URL shortener or Netflix — interviewers notice structured prep.